

手順書

μ T-Kernel で音響処理と NPU 推論を両立する音響イベント検出基盤

TRON プログラミングコンテスト 2026

福井工業高等専門学校 専攻科 辻大知

1. 概要

本書は、応募プログラムをリポジトリの取得から動作確認、紹介資料の実験 1～3 の再現まで行うための手順書である。評価は次の 3 段階のどこからでも始められる。

段階	内容	必要なもの	時間の目安
A ログから数字を作り直す (4.1 節)	紹介資料の数字を、同梱の UART ログからスクリプトで計算し直す	PC (uv)。ボード不要	約 5 分
B 実機で動作確認 (3 章)	ビルド、重みの書き込み、起動、READY と通知の確認	STM32N6570-DK、PC	約 30 分
C 実験 1～3 の再現 (4.3 節)	対照試験、長時間運転、モデル差し替え	B と同じ (通し試験だけ ESC-50 とスピーカー)	実験 1 : 10 分 × 3 実験 2 : 1 時間 実験 3 : 約 15 分

紹介資料との対応は、資料 10 章 (公開と再現) が本書 3 章、資料 4～7 章 (検証の方法と実験 1～3) が本書 4 章である。より細かい設定値とログの読み方は docs/manual.md にある。

2. 必要なもの

もの	版・入手先	用途	段階
STM32N6570-DK	1 台。外付けのハードウェアは不要	本体	B・C
USB Type-C ケーブル	1 本 (ST-LINK・電源・仮想 COM を兼ねる)	書き込みとログ	B・C
Windows PC	Windows 11 で確認	開発・ログの取得	A・B・C
STM32CubeIDE	2.2.0 (STM32CubeProgrammer 2.23.0 を同梱)	ビルド、デバッガからの実行、重みの書き込み	B・C
Git for Windows	Git Bash を使う	取得、ビルドスクリプト	A・B・C
uv	0.12 系 (Python 3.12 は uv が用意する)	集計と再生のスクリプト	A・C
ESC-10 版の重み hex	STM32N6-GettingStarted-Audio v2.3.0 の aed_weights.hex (3.3 節のコマンドで取得)	外部フラッシュに 1 回だけ書く	B・C

もの	版・入手先	用途	段階
ESC-50	git clone https://github.com/karolpiczak/ESC-50.git	通し試験（1-Ex）と、前処理の自己テストの参照	C（任意）
ヘッドホン・PC スピーカー	任意	マイクの音の確認、試験音の再生	B・C（任意）

ST Edge AI Core 4.0.1 は NPU 向けのモデルを作り直すときだけ使う。生成物はリポジトリに入っている所以、評価には要らない。

3. 実機で動作確認する（段階 B）

3.1 取得

```
git clone https://github.com/dainosuke7/mtk3-acoustic-event-detection
cd mtk3-acoustic-event-detection
git checkout submit
```

タグ submit が提出時点の状態である。

3.2 ビルド

STM32CubeIDE の File > Open Projects from File System で mtk3bsp2_stm32n657 フォルダを選び、表示される 3 つ（mtk3bsp2_stm32n657 と、その下の Appli・FSBL）をすべて開いて、FSBL と Appli をビルドする。親の mtk3bsp2_stm32n657 も開くのは、Appli のインクルードパスが親を経由するためである。2 回目からは Git Bash で次を実行してもよい。

```
bash scripts/build.sh
```

3.3 重みを外部フラッシュに書く（ESC-10 版、最初の 1 回だけ）

既定の ESC-10 版は、重み（3,282,785 バイト）を外部フラッシュの 0x70180000 から読む。重みは ST の配布物なのでリポジトリには入れていない。ST の公開リポジトリから取得して書き込む。

```
curl -L -o aed_weights.hex https://raw.githubusercontent.com/STMicroelectronics/STM32N6-GettingStarted-Audio/v2.3.0/Projects/X-CUBE-AI/models/aed_weights.hex
STM32_Programmer_CLI -c port=SWD mode=HOTPLUG -el <tools/bin>/ExternalLoader/MX66UW1G45G_STM32N6570-DK.stldr -hardRst -w aed_weights.hex
```

<tools/bin> は、STM32CubeIDE を既定の場所に入れた場合 C:\ST\STM32CubeIDE_2.2.0\STM32CubeIDE\plugins\com.st.stm32cube.ide.mcu.externaltools.cube_programmer.win32_*\tools\bin である。書き込み先の番地は hex に入っているので指定しない。

3.4 ログを取る準備

```
powershell -ExecutionPolicy Bypass -File scripts/log.ps1 -Timestamp
```

出力は logs/uart_<日時>.log になる。COM ポートが既定の COM3 と違うときは -Port COM5 のよ

うに指定する。起動の行を残すため、起動より先に立ち上げておく。

3.5 起動

1. ボードの SW1 (BOOT1) を 1-3 側 (Development boot) にする
2. USB Type-C ケーブルを CN6 (STLK) につなぐ
3. CubeIDE の Run > Debug Configurations で、起動構成 mtk3bsp2_stm32n657_FSBL Debug を実行する。Appli、FSBL の順に RAM へ書き込まれ、FSBL から実行が始まる（起動前に 2 つとも自動でビルドされる）。Appli 単体の構成では動かない
4. デバッガで一時停止していたら F8 (Resume) で続ける

3.6 確認する

起動から約 25 秒で READY が出る。READY は起動時の確認が終わった印であり、合否は次の行で見る。

ログの行	確かめていること	期待
CONFIG: A production (... model=esc10)	この起動の条件とモデル	A production、model=esc10
[probe] FNV-1a ... MATCH	外部フラッシュの重み（先頭 64 バイト）	MATCH（違えば 3.3 をやり直す）
[ram] AXISRAM6 ... MATCH [npu] ... VERSION ... MATCH	NPU の作業用 RAM・クロック・権限	MATCH
npu repeat x10 ... (bit-identical)	同じ入力で 10 回推論し、結果が同じ	10 回とも bit-identical
preproc test PASS	前処理の結果が PC と一致	PASS（参照が無ければ SKIP。4.3 節）
READY 起動確認おわり。ここから本番	起動時の確認の終わり	以後は窓ごとの行

通知の確認：犬の鳴き声を鳴らすと、平均 1.6 秒後に LCD 全面が黄色になって DOG と出る。同時に赤 LED が 1 秒点く（検出が続くあいだは点いたまま）。また、UART に JSON が 1 行出力される。緑 LED は 500 ms ごとに点滅し続け、LCD 左上の数字は 1 秒ごとに変わる。ヘッドホン (CN15) にはマイクの音が出続ける。

既定では 10 分で FINAL が出て止まる。FINAL 項で出力される audio under=0 over=0 late=0 が「音の取りこぼし 0」を示す行である。

4. 評価の手順

4.1 段階 A：同梱ログから数字を作り直す

ボードは要らない。docs/logs/ に、紹介資料の数字のもとになった UART ログを入れてある。リポジトリの直下で次のコマンドを実行する。

紹介資料の数字	コマンド	出力で見える所
実験 1 の表 (5 章)	<pre>uv run scripts/analyze_31.py docs/logs/uart_20260927_005956.log docs/logs/uart_20260927_011552.log docs/logs/uart_20260927_013950.log --out -</pre>	audio under/over/late、窓の数、log・disp の dropped
実験 2 : 1 時間で取りこぼし 0 (6-1)	<pre>uv run scripts/analyze_31.py docs/logs/uart_20260927_161828.log --out -</pre>	FINAL の audio 0/0/0、dropped 0、検出 82 件 = 80 件/時
対策前の誤報 1,216 件/時 (6-2)	<pre>uv run scripts/analyze_31.py docs/logs/uart_20260927_130250.log --out -</pre>	検出 (unknown を除く) 202 件 = 1216 件/時
規則を当てた残数 (6-2 の表)	<pre>uv run scripts/notify_replay.py docs/logs/uart_20260927_130250.log</pre>	通知 (PASS) の件数。--thr・--peak-dbfs・--rel-db で条件を変えられる
対策後の運転 (6-3)	<pre>uv run scripts/notify_replay.py docs/logs/uart_20260928_002702.log</pre>	PASS の窓の一覧と、ボードの判定との照合
実験 3 (7 章)	<pre>uv run scripts/notify_replay.py docs/logs/uart_20260928_064258.log --model fsd50k</pre>	照合の結果と通知の件数

どのログがどの報告の数字かの一覧は docs/logs/README.md にある。

4.2 段階 B : 動作確認

3 章の手順で行う。

4.3 段階 C : 実験 1~3 を再現する

変えるのは 1 行ずつで、ほかのコードは同一である。

ファイルは mtk3bsp2_stm32n657/Appli/Application/ の下にある。変えたら必ずビルドし直し、試験が終わったら既定に戻す。起動ログの CONFIG 行に条件が出るので、ログと条件を取り違えない。

実験	変える所	時間	必ず同じになるもの	環境で変わるもの
実験 1 ① 通常	既定のまま	10 分	音の取りこぼし 0/0/0、窓 642/642	—
実験 1 ② 推論を音より優先	npu/infer_task.h の INFER_PRIO_INVERT を (1)	10 分	音の取りこぼしが窓ごとに増え、入力の速さが約 14,848 Hz に落ちる。画面とログは動く	回数そのもの
実験 1 ③ 推論を 10 倍重く	npu/infer_task.h の INFER_SLOW_X を (10)	10 分	音の取りこぼし 0。窓の約 13% が消え、ログと LCD が止まる	消える窓の数
実験 2 長時間運転	audio/audio_task.c の PT_REPORT_COUNT を (7200)	1 時間	音の取りこぼし 0、窓の欠け 0、ログと画面の欠け 0	誤報の件数 (部屋の音次第)

実験	変える所	時間	必ず同じになるもの	環境で変わるもの
実験 3 モデル差し替え	npu/aed_model.h の AED_MODEL を AED_MODEL_FSD50K にし、 Clean してからビルド	10 分	CONFIG に model=fsd50k、推論 約 3.5 ms、音の取りこぼし 0.33 の重みの書き込みは要らない	誤報の件数 (部屋の音次第)

集計は、logs/ から条件ごとの最新のログを表にする場合は次の 1 行目、1 本のログを見る場合は 2 行目を使う。

```
uv run scripts/analyze_31.py --scan
uv run scripts/analyze_31.py logs/<ログ> --out -
```

誤報の件数は部屋の音で変わるので、紹介資料の値（80 件/時、3.5 件/時、250 件/時）は 4.1 節の同梱ログで確かめる。

通し試験（1-Ex、任意）：ログの取得を先に始め、PC のスピーカーで ESC-50 のクリップを決まった順に鳴らす。音量が大きすぎたり、スピーカーが近すぎたり（10 cm）するとマイクが飽和する。

```
uv run scripts/aed_play_test.py <ESC-50 のフォルダ>
uv run scripts/aed_play_test.py <ESC-50 のフォルダ> --model fsd50k
```

前処理の自己テスト（任意）：比べる音が ESC-50 由来なので、参照のヘッダはリポジトリに入っていない。次で作ってからビルドし直すと、起動時に preproc test が PASS か FAIL を出す。

```
uv run scripts/aed_clips.py <STM32N6-GettingStarted-Audio のフォルダ> <ESC-50 のフォルダ>
uv run --with tensorflow scripts/aed_clips.py --model fsd50k <ESC-50 のフォルダ>
```

5. 困ったとき

症状	原因と対処
ビルドで μ T-Kernel のヘッダ (tk/tkernel.h など) が見つからない	親の mtk3bsp2_stm32n657 を開いていない。3.2 節のとおり 3 つとも開く
デバugga が Load failed で止まる (FSD50K 版)	重みを置く AXISRAM4 の電源が入る前に書こうとしている。同梱の起動構成 mtk3bsp2_stm32n657_FSBL Debug を使う (Initialization Commands に電源投入 が入っている)
usermain に到達しない	Appli 単体の起動構成で起動している。FSBL Debug の構成を使う
[probe] が MATCH にならない	重みの hex が書けていない。3.3 節をやり直す
preproc test: SKIP と出る	前処理の参照ヘッダが無い。4.3 節の aed_clips.py で作る
log.ps1 がポートを開けない	ほかのターミナルやシリアルモニタを閉じる。-Port で COM 番号を指定する
モデルを切り替えたらビルドや動きがおかしい	Project > Clean してからビルドする (前の設定の .o が残る)

6. 既存ソフトウェアとライセンス

自作部分のライセンスは Apache License 2.0 である (LICENSE と NOTICE)。同梱した既存ソフトウェアの名称・権利者・入手先・ライセンス・改変の有無は docs/third_party.md にまとめた。 μ T-Kernel 3.0 BSP2 は T-License 2.2、ST のランタイムと生成済みネットワークは SLA0104 に従う。ESC-10 版の重み hex と ESC-50 はリポジトリに含めず、本書の手順で取得する必要がある。